

PROJECT REPORT

Solving Sudoku Puzzles using A*, Hill Climbing and Backtracking

Search Algorithms on the Kaggle Sudoku Puzzle Dataset (rohanrao/sudoku)

Field	Details
Project Title	**Solving Sudoku Puzzles using A*, Hill Climbing and Backtracking**
Course Name	**Artificial Intelligence Lab**
Student Name	**Laiba Sumaiya Nazim,** **Sumaya Sanzida,** **Masuma Tasnim Nimo**
Student ID	**23549009041,** **23549009071,** **23549009093**
Instructor Name	**Jasiya Fairiz Raisa Lecturer, Dept. of ICT, BUP** **Nazneen Akter Assistant Professor, ICT, BUP**
Submission Date	**11/11/2026**

Abstract

Project objective and problem statement. Sudoku is a constraint satisfaction problem with 81 variables and 27 all-different constraints. The objective of this project is to solve Sudoku puzzles with three algorithms from the Artificial Intelligence course (backtracking, A* search and hill climbing) and to find out which one is best when all of them receive the same puzzles.

Dataset used. The puzzles come from the Kaggle Sudoku dataset of 9 million puzzles (Rao, n.d.). A random sample of 60 puzzles is split into Easy, Medium and Hard groups of 20 by the number of blank cells. The measurements in this report were produced on a locally generated 30,000-puzzle file with the same format, because the Kaggle site could not be reached (see the data note in Section 4.1).

AI approach. Backtracking is tested in two versions, plain and with the Minimum Remaining Values (MRV) heuristic. A* search expands the partly filled board with the lowest f = g + h, where h is the total number of legal values over all empty cells, and discards dead ends. Hill climbing starts from a complete board in which every 3×3 box is valid, swaps cells inside boxes to remove repeated digits, and restarts up to 10 times.

Key results and evaluation metrics. The metrics are success rate, running time, number of states explored, peak memory and solution depth. Backtracking with MRV and A* solved all 60 puzzles in about 2.5 ms using about 100 states per puzzle. Plain backtracking solved 85 % (51 of 60) with 9,379 states on average, and hill climbing solved 1.7 % (1 of 60). A* ranks first under the ranking rule of the notebook, but it is practically tied with backtracking with MRV. The conclusion is that the algorithms that use the constraints directly are the best fit for Sudoku.

Keywords: Sudoku, A* search, Hill Climbing, Backtracking, constraint satisfaction, heuristic search, local search.

Table of Contents

Abstract **2**

1. Introduction **4**

1.1 Problem Statement 4
1.2 Purpose and Significance of the Project 4
1.3 Scope of the Project 4
1.4 Contributions 5

2. Methodology **5**

2.1 Problem Definition and State-Space Representation 5
2.2 Search Algorithm Selection and Classification 6
2.3 Algorithm Design and Implementation 6
2.4 Search Strategy and Node Exploration Process 6
2.5 Performance Evaluation Metrics 7
2.6 Tools and Technologies Used 8
2.7 Challenges Faced and Solutions Implemented 8

3. Design **8**

3.1 Problem State-Space Diagram 8
3.2 Search Algorithm Workflow / Flowchart 9
3.3 Search Tree or State-Space Graph 11
3.4 Algorithm-Specific Design and Pseudocode 12
3.5 Frontier and Explored Set Representation 13
3.6 Heuristic Function Design 13

4. Implementation **14**

4.1 Dataset Loading and Preprocessing Implementation 14
4.2 Model (Algorithm) Implementation and Training 15
4.3 Prediction (Solution) Generation 17
4.4 Code Snippets for Key Functionalities 17

5. Results **17**

5.1 Project Interface Screenshots 17
5.2 Search Results and Solution Paths 18
5.3 Performance Evaluation of Search Algorithms 18
5.4 Execution Time, Memory Usage, and Nodes Expanded 20
5.5 Path Cost and Solution Depth 22
5.6 Comparative Analysis of Search Algorithms 22
5.7 Visualization of Search Process 25

6. Conclusion **26**

6.1 Summary of the Project and Its Impact 26
6.2 Reflection on What Was Learned 26

References **27**

Appendices **27**

Appendix A. Full Code Listing of the Notebook 27
Appendix B. Supplementary Measurement Code 30
Appendix C. Reproducing the Results 31

1. Introduction

1.1 Problem Statement

Sudoku is played on a 9×9 grid. Some cells already hold a digit (the *givens*); the task is to fill every other cell with a digit from 1 to 9 so that each row, each column and each of the nine 3×3 boxes contains every digit exactly once. Seen as an AI problem, Sudoku is a *constraint satisfaction problem* (CSP) with 81 variables, a domain of nine values per variable and 27 "all different" constraints (9 rows, 9 columns, 9 boxes). The general $n^2 \times n^2$ version of the puzzle is NP-complete (Yato & Seta, 2003), so a program has to *search* for a solution rather than compute it directly.

This project solves the same set of puzzles with three algorithms taught in the Artificial Intelligence course and asks which one is best:

- **Backtracking (CSP search).** Fill empty cells one at a time with a value that does not clash with its row, column or box; when a cell has no legal value, undo the last choice and try the next one. A second version always fills the most constrained cell first (Minimum Remaining Values, MRV).
- **A* search (informed search).** Each state is a partly filled board. The algorithm always expands the state with the lowest f = g + h, where g is the number of cells filled and h is the total number of legal values left across all empty cells. States with an empty cell that has no legal value are discarded at once.
- **Hill climbing (local search).** Start with a complete but probably wrong board (every 3×3 box holds 1–9, but rows and columns contain repeats), then repeatedly swap two non-given cells inside a box when the swap reduces the number of repeated digits. If it gets stuck, restart from a new random board.

1.2 Purpose and Significance of the Project

The purpose is to compare, on identical inputs, a complete systematic method (backtracking), an informed best-first method (A*) and an incomplete local method (hill climbing), and to explain the differences in terms of what each algorithm knows about the problem. The comparison uses the same puzzles, the same correctness check and the same effort limits for every algorithm, so that differences in the results come from the algorithms and not from the experiment.

Sudoku is small enough to run hundreds of searches in seconds, yet it has the features that make real constraint problems hard: dead ends that appear only after many choices, a search space that grows exponentially with the number of blanks, and local minima. Timetabling, scheduling and resource allocation have the same structure, so what is learned here about constraint-aware search, heuristics and restarts carries over to those problems (Russell & Norvig, 2020).

1.3 Scope of the Project

- Standard 9×9 Sudoku puzzles taken from the Kaggle Sudoku dataset (Rao, n.d.); 60 puzzles are sampled and tested, 20 each in the Easy, Medium and Hard groups.
- Four algorithm configurations: plain backtracking, backtracking with MRV, A* and hill climbing with random restarts.
- Difficulty is measured by the number of blank cells, not by a human difficulty rating.

- Metrics: success rate, running time, number of states explored, peak memory and solution depth.
- Not included: advanced human-style solving techniques, puzzle generation, parallel search and a graphical interface. The project runs as a Jupyter notebook.

1.4 Contributions

1. A common board representation (a flat list of 81 numbers with precomputed rows, columns, boxes and peers) shared by all algorithms.
2. Implementations of backtracking (plain and MRV), A* with a candidate-count heuristic and dead-end pruning, and hill climbing with box-preserving swaps and random restarts.
3. A fair experiment protocol: identical puzzles for every algorithm, seeded randomness, effort caps, and an independent validity check before a run counts as solved.
4. An analysis of the A* heuristic, showing that it never underestimates the true remaining cost and is therefore not admissible (Section 3.6).
5. An evaluation by difficulty level with charts, tables of time, states explored and memory, and a ranking of the algorithms.

2. Methodology

2.1 Problem Definition and State-Space Representation

A board is stored as a flat list of 81 integers in row-major order, where 0 means “blank”. For every cell the program precomputes its 20 *peers*: the other cells in the same row, column or 3×3 box. A digit is *legal* in a cell if none of that cell’s peers already holds it. The set of legal digits of a cell is called its *candidates*. The two kinds of algorithm use different state spaces, summarised in Table 1.

Table 1. Problem formulation for the constructive algorithms and for local search.

Element	Backtracking and A* (constructive)	Hill climbing (local search)
State	A partly filled board (81 numbers, 0 = blank).	A complete board in which every 3×3 box contains 1–9 once; given cells never move.
Initial state	The puzzle as given.	Each box filled at random with its missing digits.
Actions	Put a legal digit into one empty cell.	Swap two non-given cells inside the same box.
Goal test	No empty cell is left. Only legal digits are ever placed, so all constraints hold.	Cost = 0, i.e. no row or column contains a repeated digit.
Cost	Each action costs 1, so the cost of a solution equals the number of blanks m.	Number of repeated digits over all rows and columns (to be minimised).

With m blank cells and at most nine values per cell, the constructive search space has at most 9^m states. For a puzzle with 58 blanks this naive bound is about 10^55. The local-search space is much smaller because each box is always valid: for one 58-blank puzzle used later (puzzle 6), the number of box arrangements is the product of (free cells in the box)! over the nine boxes, about 10^29.

2.2 Search Algorithm Selection and Classification

The three algorithms come from three different families of the course syllabus. They were chosen because they differ in exactly the properties that matter for a comparison: whether they are complete, whether they use a heuristic, and whether they remember a frontier of unexplored states.

Table 2. Classification of the algorithms.

Algorithm	**Family**	**Strategy**	**Uses a heuristic?**
Backtracking	CSP / uninformed	Depth-first assignment of the first empty cell, with constraint checking.	No
Backtracking + MRV	CSP / heuristic ordering	Depth-first, but always fills the cell with the fewest legal values (fail-first principle; Haralick & Elliott, 1980).	Yes (variable ordering)
A*	Informed search	Best-first: expands the state with the lowest f = g + h from a priority queue.	Yes (h = total candidates)
Hill climbing	Local search	Iterative improvement of one complete board by best swaps; random restarts.	Cost function only

2.3 Algorithm Design and Implementation

Backtracking

The recursive function counts every value it tries (*nodes*). In the plain version it fills the first empty cell; in the MRV version it fills the empty cell with the fewest candidates, so contradictions are found early. After all values of a cell fail, the cell is reset to 0 and the function returns to the previous cell. A cap of 50,000 assignments stops runs that take too long; a run that reaches the cap is counted as not solved.

A* search

A helper function, analyse, scans a board once and returns three things: the heuristic value h (the sum of the number of candidates over all empty cells), the empty cell with the fewest candidates, and its candidates. It returns nothing if some empty cell has no candidate, which marks a dead end. The priority queue is a binary heap of tuples (f, tie-breaker, g, board, info). The children of a state are created by filling the cell with the fewest candidates with each of its candidates, so the branching factor is always as small as the board allows. The search stops after 3,000 expansions.

Hill climbing

Every 3×3 box is first filled with its missing digits in random order, so boxes are always valid and only rows and columns can contain repeats. The cost is the number of repeated digits over all rows and columns. A move swaps two non-given cells inside one box. In each step the boxes are examined in random order, and in the first box that offers an improving swap the best swap of that box is applied. If no box offers an improvement the board is at a local minimum and the algorithm restarts from a new random board, up to 10 restarts with at most 300 steps each.

2.4 Search Strategy and Node Exploration Process

The algorithms differ in the order in which they visit states and in what is counted as a *state explored* (nodes in the code):

- **Backtracking** explores depth-first. At each level it tries the candidate values of one cell in increasing order, and every value tried counts as one node.
- **A*** explores best-first. It repeatedly pops the state with the lowest f; if two states have the same f, the one inserted earlier is popped first. Every child that is generated counts as one node, including children discarded as dead ends. A state is *expanded* when it is popped and its children are generated.
- **Hill climbing** has no tree. It examines swaps, and every swap whose effect on the cost is evaluated counts as one node.

Table 7 in Section 3.5 shows the first six expansions of A* on a real puzzle, and Figure 6 in Section 3.3 shows the first two levels of its search tree.

2.5 Performance Evaluation Metrics

The theoretical properties of the four algorithms are summarised in Table 3. Here *m* is the number of blank cells and 9 is the largest possible number of candidates for a cell.

Table 3. Theoretical properties of the algorithms as implemented.

Property	Backtracking (plain / MRV)	A*	Hill climbing
Complete?	Yes without a cap (it is exhaustive). With the 50,000-assignment cap it can give up.	Yes in a finite space. With the 3,000-expansion cap it can give up.	No: it can stop at a local minimum.
Optimal?	Not applicable: every solution fills exactly m cells, so any valid completion is equally good.	Every solution has cost m, so the first goal found is optimal in path cost. The heuristic is not admissible (Section 3.6), but this does not matter here.	Not applicable: there is no path, only a final board.
Time	O(9^m) worst case. MRV has the same bound but a far smaller tree in practice; each node costs O(m) candidate checks.	O(9^m) worst case; each generated state costs one pass over all empty cells (analyse).	Each step evaluates at most 9 boxes × 36 swaps; at most 10 restarts × 300 steps.
Space	O(m) recursion depth.	O(9^m) worst case, because all generated states wait in the heap.	O(1) beyond the board of 81 cells.

The measured metrics were collected for every puzzle and every algorithm:

- **Success rate:** the percentage of puzzles solved. A run counts as solved only if the final board is a valid Sudoku (all nine rows, columns and boxes contain 1–9) and keeps every given digit.
- **Running time:** wall-clock seconds measured with time.perf_counter(); the mean, median and maximum are reported.
- **States explored:** the node counts defined in Section 2.4.
- **Peak memory:** the peak of Python allocations during one run, measured with tracemalloc in a supplementary run (Appendix B).
- **Solution depth and path cost:** the number of cells filled to reach the goal (equal to the number of blanks).

2.6 Tools and Technologies Used

Table 4. Tools and libraries.

Tool	Use in this project
Python 3 and Jupyter Notebook	Implementation of all algorithms, experiments and charts.
time	perf_counter() for timing each run.
heapq	Binary heap that serves as the priority queue of A*.
random	Seeded random numbers: puzzle sampling, random initial boards and random box order in hill climbing.
NumPy	Imported in the setup cell; the final code does not need it.
pandas	Reading the 9-million-row CSV with row skipping, sampling, difficulty groups, grouping and pivot tables.
matplotlib	Bar charts, box plot and scatter plot of the results.
tracemalloc (standard library)	Peak memory per run in the supplementary measurement (Appendix B).
Kaggle Sudoku dataset	Source of the puzzles (Rao, n.d.).

2.7 Challenges Faced and Solutions Implemented

Table 5. Challenges and the solutions used in the notebook.

Challenge	Solution
The dataset has 9 million rows; loading all of it is slow and memory-hungry.	Rows are skipped while reading (each kept with probability 0.3 %, seed 42), so the sample is spread over the whole file; 60 puzzles are then drawn.
Blank cells appear as 0 or as “.”.	Every “.” is replaced by 0 before the puzzle is converted to a list.
The file gives no difficulty rating.	The number of blanks is used. The sample is split into Easy, Medium and Hard thirds of equal size.
Plain backtracking can run for a very long time on some puzzles.	A cap of 50,000 assignments. The MRV version is added as a second backtracking variant.
A* recomputes the heuristic for every state and can store many boards.	One scan (analyse) returns h, the branching cell and its candidates; dead ends are discarded before they enter the heap; 3,000-expansion cap.
Hill climbing gets trapped in local minima.	Box-preserving swaps keep every box valid, and up to 10 random restarts give it more chances.
A fair comparison between very different algorithms.	All algorithms receive the same puzzles; hill climbing is seeded; is_valid checks the final board independently of the algorithm.
Running times differ by orders of magnitude.	Time and node charts use a logarithmic axis.

3. Design

3.1 Problem State-Space Diagram

Figure 1 shows the state representation on a real puzzle from the sample (puzzle 6, Hard, 58 blanks). Panel (a) is the initial state. Panel (b) highlights cell 23 (row 2, column 5) and its peers. Row 2 already contains 5, 9, 1 and 6, column 5 contains 3, and the box contains 1, 9, 8 and 6. The digits in use are therefore {1, 3, 5, 6, 8, 9}, which leaves {2, 4, 7} as the legal values. Each

of these three values leads to a different successor state. The goal state is reached after 58 such steps (Section 5.2).

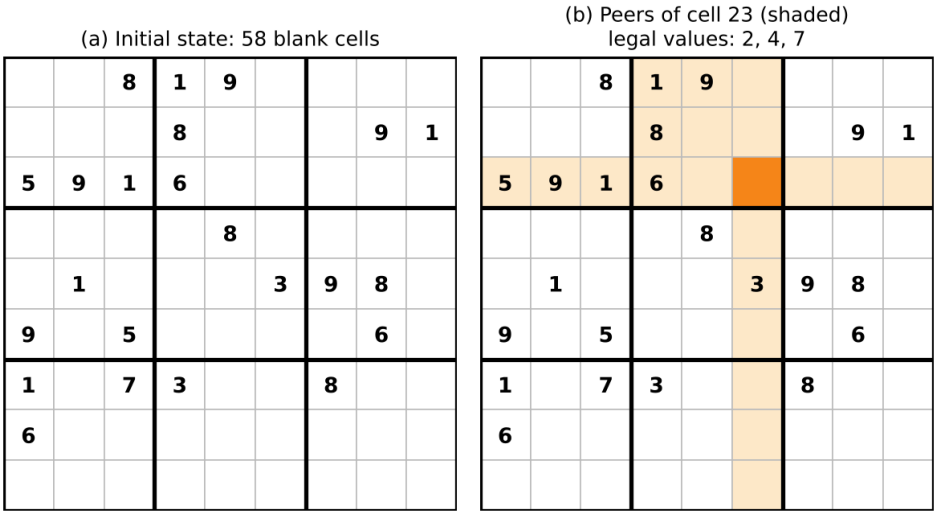


Figure 1. State representation: (a) initial state of puzzle 6 and (b) the peers and legal values of cell 23.

3.2 Search Algorithm Workflow / Flowchart

Figure 2 shows the workflow of the whole experiment. Figure 3, Figure 4 and Figure 5 show the flow of control of the three algorithms.

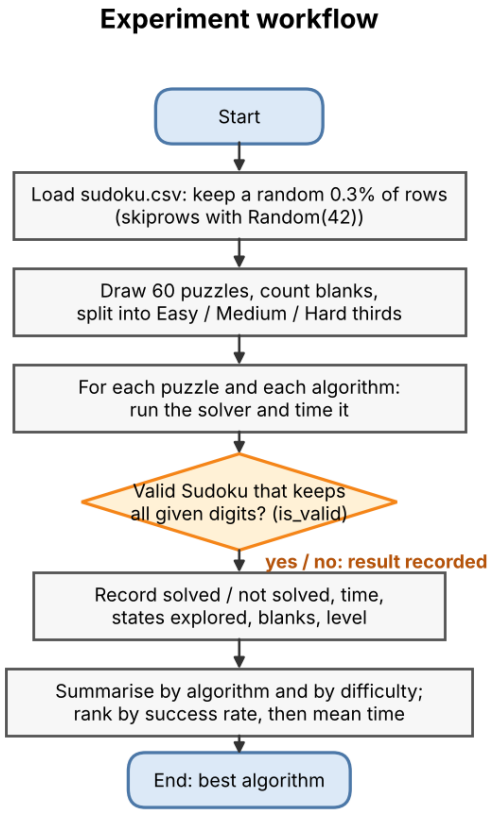


Figure 2. Workflow of the experiment.

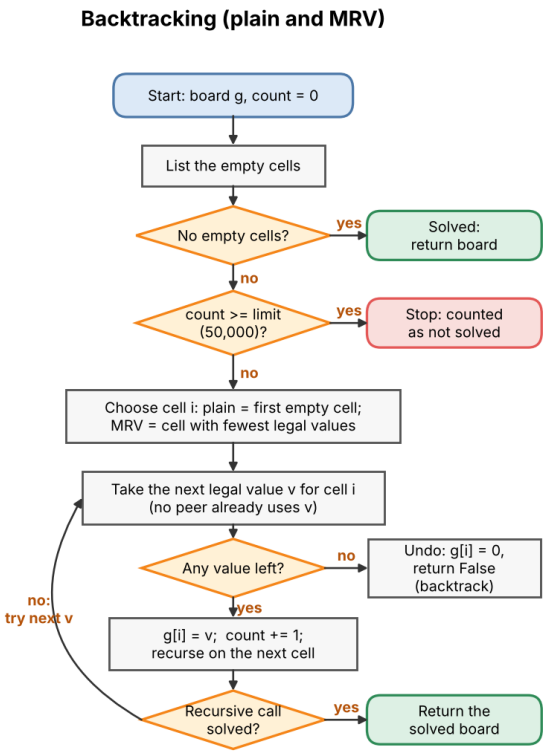


Figure 3. Flowchart of backtracking (plain and MRV).

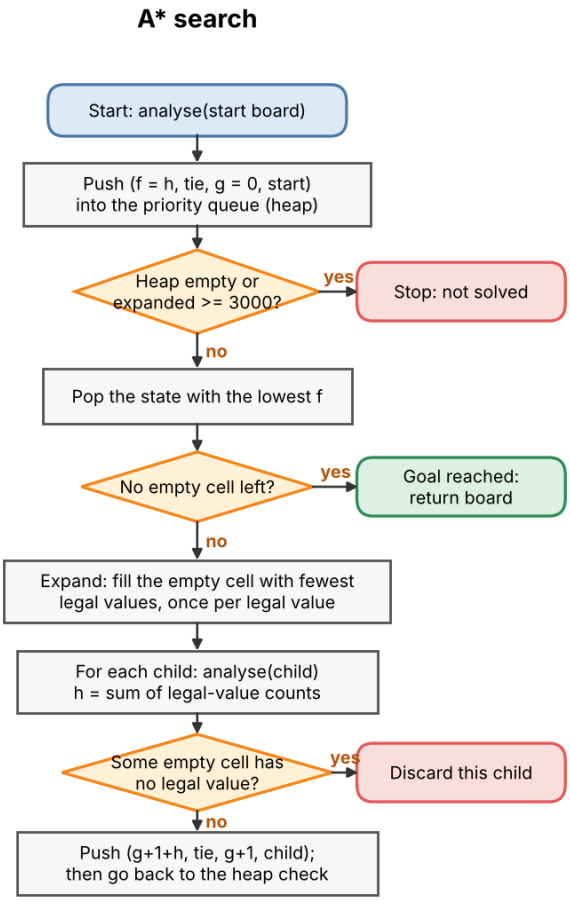


Figure 4. Flowchart of A* search.

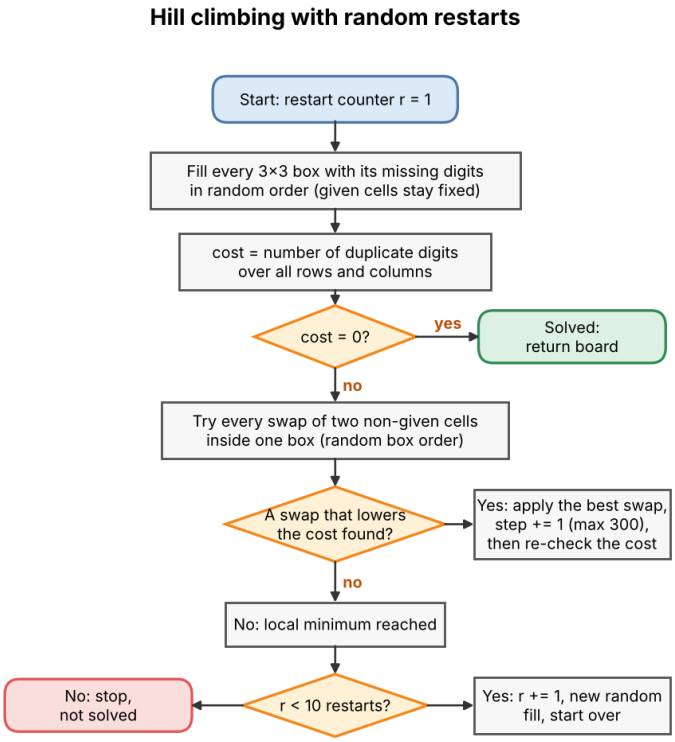


Figure 5. Flowchart of hill climbing with random restarts.

3.3 Search Tree or State-Space Graph

Figure 6 shows the first two levels of the A* search tree for puzzle 6. The values were produced by the project's own analyse function. The root has h = 280 and branches on cell 23, which has the fewest candidates (2, 4 and 7). Two children have f = 266 and one has f = 267. Because the child with value 2 was inserted first, it is expanded first. Its own children fill cell 5, which now has three candidates (4, 5, 7) and f values 251, 252 and 255. The f values fall quickly because every filled cell removes one blank (g rises by 1) and removes many candidates from the peers (h falls by much more).

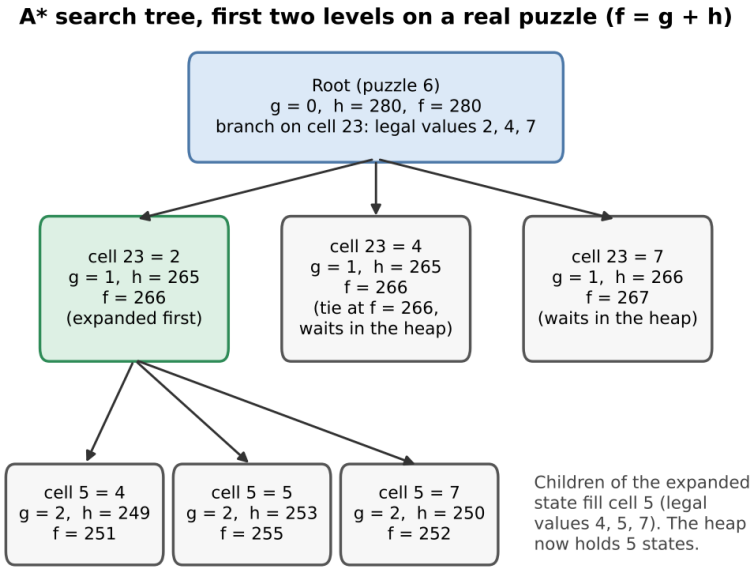


Figure 6. First two levels of the A* search tree for puzzle 6 (f = g + h).

3.4 Algorithm-Specific Design and Pseudocode

The pseudocode below follows the implementation. LEGAL(board, i) returns the digits 1–9 that none of the 20 peers of cell i uses.

Algorithm 1: Backtracking (plain and MRV)

```
BACKTRACK(puzzle, mrv, limit = 50000)
    board <- copy of puzzle;  count <- 0
    SOLVE():
        E <- empty cells of board
        if E is empty:           return TRUE
        if count >= limit:       return FALSE
        i <- (mrv) ? cell in E with fewest LEGAL values : first cell of E
        for each v in LEGAL(board, i):
            board[i] <- v;  count <- count + 1
            if SOLVE():          return TRUE
        board[i] <- 0
        return FALSE
    return SOLVE(), count, board
```

Algorithm 2: A search*

```
ANALYSE(board)
    h <- 0;  best <- none
    for each empty cell i:
        c <- LEGAL(board, i)
        if c is empty:           return DEAD-END
        h <- h + |c|
        if best is none or |c| < |best.candidates|:  best <- (i, c)
    return (h, best)


ASTAR(puzzle, limit = 3000)
    info <- ANALYSE(puzzle);  if DEAD-END: return FAIL
    heap <- { (f = h, tie = 0, g = 0, puzzle, info) };  expanded <- 0
    while heap is not empty and expanded < limit:
        (f, tie, g, state, info) <- pop entry with lowest f (ties: lowest tie)
        if state has no empty cell:   return state
        expanded <- expanded + 1
        for each v in info.candidates:
            child <- state with info.cell = v;  nodes <- nodes + 1
            ci <- ANALYSE(child)
            if ci is DEAD-END:       continue
            push (g + 1 + ci.h, ++tie, g + 1, child, ci)
    return FAIL
```

Algorithm 3: Hill climbing with random restarts

```
HILL-CLIMBING(puzzle, restarts = 10, max_steps = 300)
    for r <- 1 to restarts:
        board <- puzzle;  fill every 3x3 box with its missing digits, random order
        cost <- number of repeated digits over all rows and columns
        for step <- 1 to max_steps:
            if cost = 0:         return board
            for each box in random order:
                find the swap of two non-given cells in the box with the
                largest cost reduction
                if such a swap exists: apply it;  update cost;  next step
            if no box had an improving swap:  break      // local minimum
        if cost = 0:             return board
    return FAIL
```

3.5 Frontier and Explored Set Representation

Table 6. How each algorithm stores the states it has seen and the states it still has to visit.

Algorithm	Frontier	Explored set
Backtracking	The recursion stack. One board is modified in place, and each assignment is undone on return.	None. Undoing assignments keeps memory at O(m).
A*	A min-heap of tuples (f, tie-breaker, g, board, info). The tie-breaker is an increasing counter, so equal f values are served in insertion order and boards are never compared.	None. A state always fills the same cell, so the same board is rarely reached by two different routes.
Hill climbing	None: only the current board and the best swap found.	None. Random restarts replace memory of earlier boards.

Table 7 is a real trace of the A* heap on puzzle 6. It lists the first six expansions: the state popped, the cell it fills, the children pushed with their f values, and the size of the heap afterwards. From step 4 on, the filled cells have a single candidate, so the heap stops growing.

Table 7. First six A* expansions on puzzle 6 (heap contents as produced by the implementation).

Step	Popped (f, g)	Cell filled	Legal values	Children pushed (value: f)	Heap size
1	(280, 0)	23	2, 4, 7	2: 266; 4: 266; 7: 267	3
2	(266, 1)	5	4, 5, 7	4: 251; 5: 255; 7: 252	5
3	(251, 2)	14	5, 7	5: 244; 7: 241	6
4	(241, 3)	22	3	3: 237	6
5	(237, 4)	13	5	5: 232	6
6	(232, 5)	50	1	1: 227	6

3.6 Heuristic Function Design

The heuristic is defined for a partly filled board n:

- **g(n)** = number of cells filled so far (each action costs 1).
- **h(n)** = the sum, over all empty cells, of the number of legal values of that cell.
- **f(n) = g(n) + h(n)**; the state with the lowest f is expanded first.
- **Dead end:** if some empty cell has no legal value, the state cannot be completed and is discarded.

Why it guides the search. A cell with a single legal value contributes 1 to h, which is exactly what filling it costs. A cell with many candidates contributes more, so boards with many nearly decided cells have a lower f and are explored first. In this way the heuristic combines the "most constrained first" idea of MRV with a global view of how constrained the whole board is.

Admissibility. A heuristic is admissible if it never overestimates the true remaining cost. Here the true remaining cost of a solvable board is exactly e, the number of empty cells, because every solution fills each empty cell once. Every non-dead-end empty cell has at least one candidate, so $h = \Sigma|\text{candidates}| \geq e$. The heuristic therefore overestimates by $\Sigma(|\text{candidates}| - 1)$ and is equal to the true cost only when every empty cell has a single legal value. At the root of puzzle 6, h = 280 while the true remaining cost is 58, an overestimate of 222. The heuristic is not admissible, so the usual optimality guarantee of A* does not hold. This is acceptable here

because all solutions have the same cost m: along any solution path, g + e = m, and the f value of the goal equals the number of blanks (f = 58 for puzzle 6, as the last point of Figure 13 shows). The heuristic is therefore used only as a guide for the order of expansion.

4. Implementation

4.1 Dataset Loading and Preprocessing Implementation

The first two columns of sudoku.csv hold the puzzle and its solution as 81-character strings. Because the file has 9 million rows, pd.read_csv is given a skiprows function that keeps each row with probability 0.3 % using a seeded random generator, so the sample is spread over the whole file. 60 puzzles are then drawn, blanks are counted, and the puzzles are split by rank of blank count into three equal groups.

```
N_PUZZLES=60
KEEP=0.003
rng=random.Random(42)
df=pd.read_csv('sudoku.csv',dtype=str,skiprows=lambda i:i>0 and rng.random()>KEEP)
df=df.rename(columns={df.columns[0]:'quizzes',df.columns[1]:'solutions'})
df['quizzes']=df['quizzes'].str.replace('.','0',regex=False)
sample=df.sample(N_PUZZLES,random_state=42).reset_index(drop=True)
sample['blanks']=sample['quizzes'].str.count('0')
sample['level']=pd.qcut(sample['blanks'].rank(method='first'),3,labels=['Easy','Medium','Hard']).astype(str)
sample.groupby('level')['blanks'].agg(['count','min','max','mean'])
```

The board helpers precompute the rows, columns, boxes and the 20 peers of every cell. candidates(g, i) returns the legal digits of cell i and to_grid turns a puzzle string into a list of integers.

```
ROWS=[[r*9+c for c in range(9)] for r in range(9)]
COLS=[[r*9+c for r in range(9)] for c in range(9)]
BOXES=[[(br*3+i)*9+bc*3+j for i in range(3) for j in range(3)] for br in range(3) for bc in range(3)]
PEERS=[[j for j in range(81) if j!=i and (j//9==i//9 or j%9==i%9 or (j//27==i//27 and (j%9)//3==(i%9)//3))] for i in range(81)]
def candidates(g,i):
    used={g[j] for j in PEERS[i]}
    return [v for v in range(1,10) if v not in used]
def to_grid(s):
    return [int(ch) for ch in s]
```

Table 8. Composition of the 60-puzzle sample by difficulty (number of blank cells).

Level	Puzzles	Fewest blanks	Most blanks	Mean blanks
Easy	20	45	50	47.2
Medium	20	50	54	52.6
Hard	20	54	58	56.9

Data note. The Kaggle website could not be reached from the environment in which this report was produced. The notebook was therefore executed unchanged on a locally generated file with the same format as sudoku.csv (columns *quizzes* and *solutions*): 30,000 puzzles created by shuffling a valid Sudoku pattern and blanking between 45 and 58 random cells. The notebook's 0.3 % sampling kept 88 rows, from which the 60 puzzles were drawn. These puzzles are not guaranteed to have a unique solution (A* found a solution for puzzle 6 that differs from the stored one). Absolute times and success rates will therefore differ when the notebook is run on the original Kaggle file; the method and the structure of the analysis do not change.

4.2 Model (Algorithm) Implementation and Training

The algorithms are search procedures, so nothing is trained and no parameters are learned. Table 9 lists the settings that control the experiment.

Table 9. Configuration parameters.

Parameter	Value	Meaning
N_PUZZLES	60	Number of puzzles tested (20 per difficulty level).
KEEP	0.003	Fraction of CSV rows kept while loading.
Random seed	42	Used for sampling and for hill climbing.
Backtracking limit	50,000	Maximum number of assignments tried per puzzle.
A* limit	3,000	Maximum number of expanded states per puzzle.
Hill climbing restarts	10	Number of random restarts.
Hill climbing max steps	300	Maximum accepted swaps per restart.

Backtracking (plain and MRV)

```
def backtracking(puzzle,mrv=False,limit=50000):
    g=list(puzzle)
    count=[0]
    def solve():
        empties=[i for i in range(81) if g[i]==0]
        if not empties:
            return True
        if count[0]>=limit:
            return False
        i=min(empties,key=lambda k:len(candidates(g,k))) if mrv else empties[0]
        for v in candidates(g,i):
            g[i]=v
            count[0]+=1
            if solve():
                return True
        g[i]=0
        return False
    ok=solve()
    return ok,count[0],g
```

A search*

```
def analyse(g):
    total=0
    best=None
    best_c=None
    for i in range(81):
        if g[i]==0:
            c=candidates(g,i)
            if not c:
                return None
            total+=len(c)
            if best is None or len(c)<len(best_c):
                best,best_c=i,c
    return total,best,best_c
def astar(puzzle,limit=3000):
    start=tuple(puzzle)
    info=analyse(start)
    if info is None:
        return False,0,list(start)
    tie=0
    heap=[(info[0],tie,0,start,info)]
    nodes=0
    expanded=0
    while heap and expanded<limit:
        f,_,g,state,info=heapq.heappop(heap)
        if info[1] is None:
            return True,nodes,list(state)
        expanded+=1
        for v in info[2]:
            child=list(state)
            child[info[1]]=v
            nodes+=1
```

```
            ci=analyse(child)
            if ci is None:
                continue
            tie+=1
            heapq.heappush(heap,(g+1+ci[0],tie,g+1,tuple(child),ci))
    return False,nodes,list(start)
```

Hill climbing

```
def line_cost(g,idx):
    return 9-len({g[k] for k in idx})
def total_cost(g):
    return sum(line_cost(g,l) for l in ROWS+COLS)
def hill_climbing(puzzle,restarts=10,max_steps=300):
    free=[[k for k in box if puzzle[k]==0] for box in BOXES]
    evaluated=0
    g=list(puzzle)
    for _ in range(restarts):
        g=list(puzzle)
        for b,box in enumerate(BOXES):
            missing=[v for v in range(1,10) if v not in {g[k] for k in box}]
            random.shuffle(missing)
            for k in free[b]:
                g[k]=missing.pop()
        cost=total_cost(g)
        for _ in range(max_steps):
            if cost==0:
                return True,evaluated,g
            moved=False
            for b in random.sample(range(9),9):
                f=free[b]
                best=0
                move=None
                for x in range(len(f)):
                    for y in range(x+1,len(f)):
                        a,c=f[x],f[y]
                        rows={a//9,c//9}
                        cols={a%9,c%9}
                        before=sum(line_cost(g,ROWS[r]) for r in rows)+sum(line_cost(g,COLS[k]) for k in cols)
                        g[a],g[c]=g[c],g[a]
                        after=sum(line_cost(g,ROWS[r]) for r in rows)+sum(line_cost(g,COLS[k]) for k in cols)
                        g[a],g[c]=g[c],g[a]
                        evaluated+=1
                        if after-before<best:
                            best=after-before
                            move=(a,c)
                if move:
                    a,c=move
                    g[a],g[c]=g[c],g[a]
                    cost+=best
                    moved=True
                    break
            if not moved:
                break
        if cost==0:
            return True,evaluated,g
    return False,evaluated,g
```

4.3 Prediction (Solution) Generation

Every algorithm receives exactly the same puzzles. The function `is_valid` checks that the returned board has the digits 1–9 once in every row, column and box and keeps all given digits; only then is the run recorded as solved. For each run the loop stores the time, the number of states explored, the number of blanks and the difficulty level. The `summary` table is then built with `groupby`.

```
def is_valid(grid,puzzle):
    full=set(range(1,10))
    return all(set(grid[k] for k in line)==full for line in ROWS+COLS+BOXES) and all(p==0 or p==v for p,v in zip(puzzle,grid))
random.seed(42)
algorithms={
    'Backtracking':lambda p:backtracking(p,mrv=False),
    'Backtracking + MRV':lambda p:backtracking(p,mrv=True),
    'A*':astar,
    'Hill Climbing':hill_climbing
}
records=[]
```

```
for idx,row in sample.iterrows():
    puzzle=to_grid(row['quizzes'])
    for name,fn in algorithms.items():
        t=time.perf_counter()
        ok,nodes,grid=fn(puzzle)
        elapsed=time.perf_counter()-t
        records.append({'puzzle':idx,'algorithm':name,'solved':bool(ok and
is_valid(grid,puzzle)),'time':elapsed,'nodes':nodes,'blanks':row['blanks'],'level':row['level']})
results=pd.DataFrame(records)
summary=results.groupby('algorithm').agg(success_rate=('solved',lambda
s:s.mean()*100),mean_time=('time','mean'),median_time=('time','median'),max_time=('time','max'),mean_nodes=('nodes','mean')).rein
dex(list(algorithms))
summary.round(4)
```

4.4 Code Snippets for Key Functionalities

Table 10 maps each key functionality to its function in the notebook. The ranking code below it applies the selection rule of the notebook: correctness first (success rate), then the lower mean time.

Table 10. Key functionalities and where they are implemented.

Functionality	Function / code	Notebook section
Sample puzzles from the CSV	pd.read_csv(..., skiprows=...), df.sample	1. Loading the data
Legal values of a cell	candidates(g, i) with PEERS	2. Board helpers
Constraint-based search	backtracking(puzzle, mrv, limit)	3. Backtracking
Heuristic and dead-end detection	analyse(g)	4. A* Search
Informed search	astar(puzzle, limit)	4. A* Search
Conflict count (objective)	line_cost, total_cost	5. Hill Climbing
Local search with restarts	hill_climbing(puzzle, restarts, max_steps)	5. Hill Climbing
Independent correctness check	is_valid(grid, puzzle)	6. Running the experiment
Ranking of the algorithms	summary.sort_values(...)	10. Picking the best algorithm

```
ranking=summary.sort_values(['success_rate','mean_time'],ascending=[False,True])
ranking['rank']=range(1,len(ranking)+1)
display(ranking[['rank','success_rate','mean_time','mean_nodes']].round(4))
print('Best algorithm:',ranking.index[0])
```

5. Results

All numbers in this section were measured by running the notebook code on the 60-puzzle sample described in Section 4.1 (see the data note there). The three difficulty groups contain 20 puzzles each, so every algorithm was run 60 times and the experiment contains 240 runs.

5.1 Project Interface Screenshots

Not applicable. The project has no graphical interface; it runs as a Jupyter notebook and its output consists of tables and charts, which are shown in Sections 5.3 to 5.7.

5.2 Search Results and Solution Paths

Figure 7 shows puzzle 6 and the solution found by A*. The solution is a valid Sudoku that keeps all 23 given digits. A* needed 59 expansions and generated 77 states to fill 58 cells, so only one expansion was spent off the final solution path. The largest number of states waiting

in the heap at any time was 18. The solution path starts as listed in Table 7: cell 23 receives the value 2, then cell 5 the value 4, then cell 14 the value 7; the next three cells (22, 13 and 50) each have a single legal value.

(a) Initial state

		8	1	9				
			8				9	1
5	9	1	6					
				8				
	1				3	9	8	
9		5					6	
1		7	3			8		
6								

(b) Goal state found by A* (blue = filled in)

2	7	8	1	9	4	5	3	6
4	3	6	8	5	7	2	9	1
5	9	1	6	3	2	7	4	8
3	6	4	9	8	5	1	7	2
7	1	2	4	6	3	9	8	5
9	8	5	2	7	1	4	6	3
1	5	7	3	4	6	8	2	9
6	2	9	7	1	8	3	5	4
8	4	3	5	2	9	6	1	7

Figure 7. Puzzle 6: (a) initial state and (b) goal state found by A* (blue digits were filled in by the search).

Plain backtracking, backtracking with MRV and A* returned a valid board in every run that finished, which is the meaning of “solved” in this report. Hill climbing returned a valid board for only 1 of the 60 puzzles (an Easy puzzle with 45 blanks). For the others it ended in a local minimum; in an illustrative run on puzzle 6 the ten restarts stopped at costs between 6 and 16 repeated digits (Figure 13).

5.3 Performance Evaluation of Search Algorithms

Table 11. Overall results on the 60 puzzles. Times are in milliseconds.

Algorithm	Solved	Success rate	Mean time	Median time	Max time	Mean states
Backtracking	51 / 60	85.0 %	21.73	2.21	132.29	9,379
Backtracking + MRV	60 / 60	100.0 %	2.50	1.45	35.63	95
A*	60 / 60	100.0 %	2.47	1.63	18.53	102
Hill Climbing	1 / 60	1.7 %	23.12	21.81	48.80	6,418

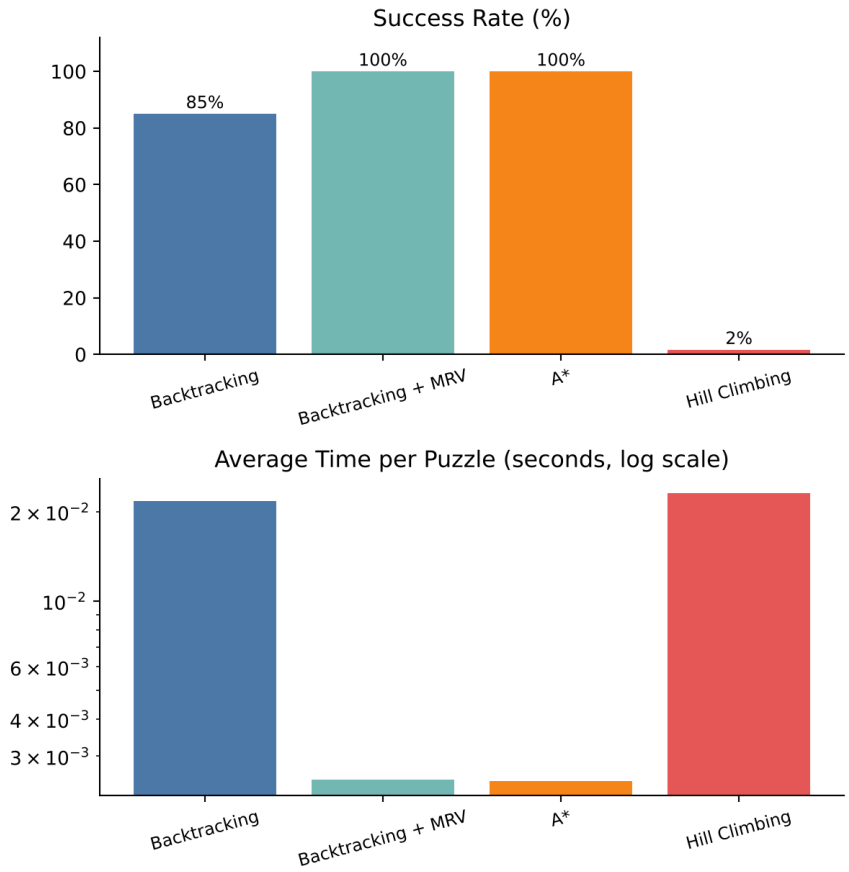


Figure 8. Success rate (top) and average time per puzzle (bottom, log scale).

Backtracking with MRV and A* solved all 60 puzzles. Plain backtracking solved 51 (85 %); its 9 failures all stopped at the 50,000-assignment cap (50,014 to 50,019 assignments), so they are cases where the search gave up, not proof that those puzzles are unsolvable. Hill climbing solved 1 puzzle (1.7 %). In time, MRV and A* are about nine times faster on average than plain backtracking (2.5 ms against 21.7 ms), and hill climbing is the slowest per puzzle (23.1 ms) because a failed run always uses all of its restarts.

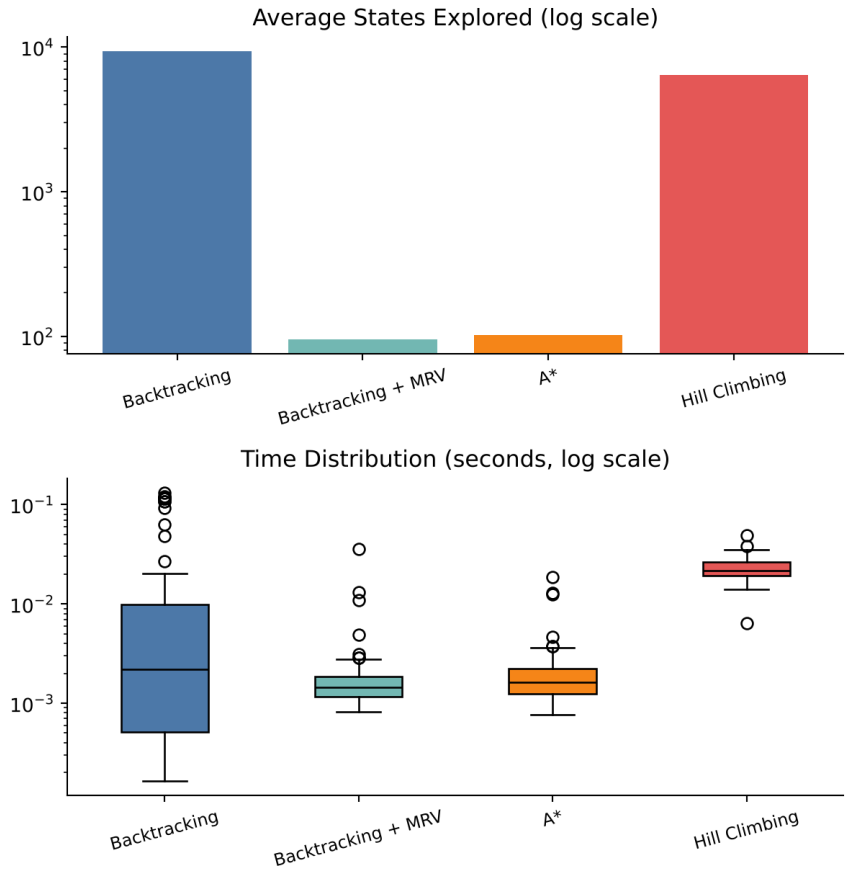


Figure 9. Average number of states explored (top, log scale) and distribution of running times (bottom, log scale).

The number of states explains the timing. MRV generated on average 95 assignments per puzzle and A* 102 states, against 9,379 for plain backtracking and 6,418 swap evaluations for hill climbing. The box plot also shows how predictable each algorithm is: plain backtracking has the widest spread of running times (standard deviation 40.9 ms) and A* the narrowest (3.0 ms; MRV 4.8 ms, hill climbing 6.5 ms).

5.4 Execution Time, Memory Usage, and Nodes Expanded

Table 12 breaks the results down by difficulty. Plain backtracking becomes both slower and less reliable as the number of blanks grows. MRV and A* stay at 100 % and need only about 52 to 137 states per puzzle, even for Hard puzzles.

Table 12. Results by difficulty level: success rate (%), mean time (ms) and mean number of states explored.

Level	Algorithm	Success (%)	Mean time (ms)	Mean states
Easy	Backtracking	95.0	7.31	3,130
	Backtracking + MRV	100.0	1.19	52
	A*	100.0	1.29	60
	Hill Climbing	5.0	17.88	4,921
Medium	Backtracking	80.0	28.29	12,901
	Backtracking + MRV	100.0	2.34	97
	A*	100.0	2.74	112
	Hill Climbing	0.0	23.93	6,458
Hard	Backtracking	80.0	29.59	12,106
	Backtracking + MRV	100.0	3.97	137
	A*	100.0	3.38	135
	Hill Climbing	0.0	27.54	7,875

Memory. The notebook does not measure memory, so peak memory was measured in a supplementary run of the same 60 puzzles with tracemalloc (Appendix B). It counts Python allocations only. Table 13 and Figure 10 show the result.

Table 13. Peak memory per puzzle (KB of Python allocations): overall and by difficulty.

Algorithm	Mean	Maximum	Easy	Medium	Hard
Backtracking	19.4	23.7	16.4	19.4	22.4
Backtracking + MRV	20.0	23.8	16.8	20.1	23.0
A*	8.4	18.0	5.4	7.9	11.9
Hill Climbing	4.0	4.2	4.0	4.0	4.1

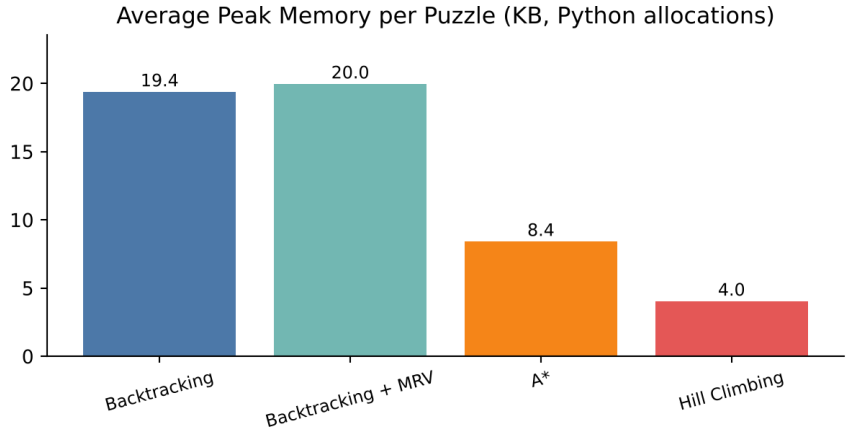


Figure 10. Average peak memory per puzzle (KB, Python allocations only).

Hill climbing uses the least memory (4 KB: one board and a few counters). A* uses about 8 KB on average and grows with difficulty (5 KB for Easy, 12 KB for Hard) because more states wait in the heap. Both backtracking versions peak at about 19 to 20 KB. This is higher than expected for a method that stores only one board: every recursive call keeps a list of the

remaining empty cells, and the recursion is up to 58 levels deep. The notebook's conclusion expects A* to be memory-hungry; on these puzzles the heuristic keeps the heap very small (at most 18 states on puzzle 6), so this expectation was not observed. It would become relevant on puzzles that force A* to expand many more states.

Table 14. Time per state explored (mean time divided by mean number of states, microseconds).

Algorithm	Mean time (ms)	Mean states	Time per state (µs)
Backtracking	21.73	9,379	2.3
Backtracking + MRV	2.50	95	26.3
A*	2.47	102	24.2
Hill Climbing	23.12	6,418	3.6

A plain backtracking step is cheap (about 2 µs) because it checks only one cell, but it takes about 99 times more steps than MRV. MRV and A* pay about 25 µs per state because they recompute the candidates of every empty cell, which is the price of their much smaller trees.

5.5 Path Cost and Solution Depth

In the constructive algorithms every action fills one cell at cost 1, so the solution depth and the path cost are both equal to the number of blanks: 47.2 on average for Easy, 52.6 for Medium and 56.9 for Hard puzzles (Table 8), and 52.2 over the whole sample. All solutions of a puzzle have the same cost, so "optimal" and "solved" coincide. Hill climbing has no path; its answer is the final board. Table 15 compares the number of states each algorithm generates per filled cell. A perfect search that never takes a wrong step would need exactly 1.0.

Table 15. Solution depth (= path cost) and states explored per filled cell.

Level	Solution depth = path cost	Backtracking	Backtracking + MRV	A*
Easy	47.2	66.3	1.11	1.28
Medium	52.6	245.3	1.84	2.12
Hard	56.9	212.8	2.40	2.37
All	52.2	179.6	1.82	1.96

MRV and A* generate fewer than two states for every cell they fill, which means they almost never go down a wrong branch. Plain backtracking generates about 180 assignments per filled cell on average (this figure includes the runs that hit the cap).

5.6 Comparative Analysis of Search Algorithms

Table 16 reproduces the ranking rule of the notebook: algorithms are ordered by success rate and ties are broken by the lower mean time.

Table 16. Ranking of the algorithms (notebook section 10).

Rank	Algorithm	Success rate (%)	Mean time (ms)	Mean states
1	A*	100.0	2.47	102
2	Backtracking + MRV	100.0	2.50	95
3	Backtracking	85.0	21.73	9,379
4	Hill Climbing	1.7	23.12	6,418

By this rule **A*** is ranked first and is the best algorithm of this run. A* and backtracking with MRV are, however, practically tied: both solved 100 % of the puzzles, their mean times differ by 0.03 ms (2.47 against 2.50 ms) and MRV even has a slightly lower median time and a slightly lower number of states. To see whether the order is stable, the timing of the two algorithms was repeated eight times on the same puzzles (Appendix B; Table 17). A* was faster in five repetitions, MRV in two, and one was a tie, always within about 0.2 ms. The first place of A* is therefore not a reliable difference.

Table 17. Mean time per puzzle (ms) in eight repeated timing runs of MRV and A*.

Run	Backtracking + MRV	A*	Faster
1	2.34	2.34	tie
2	2.55	2.54	A*
3	2.21	2.16	A*
4	2.19	2.17	A*
5	2.16	2.12	A*
6	2.69	2.72	MRV
7	3.08	2.95	A*
8	3.12	3.32	MRV

Effect of difficulty. Figure 11 shows how the algorithms react to more blanks. Plain backtracking drops from 95 % (Easy) to 80 % (Medium and Hard) and its mean time rises from 7.3 ms to about 29 ms. MRV and A* stay at 100 %, and their mean times rise only from about 1.2 to 1.3 ms (Easy) to about 3.4 to 4.0 ms (Hard). Hill climbing solves one Easy puzzle and none of the Medium and Hard ones. Figure 12 plots the time of every run against the number of blanks; the correlation between time and blanks is 0.63 for hill climbing, 0.33 for A*, 0.25 for plain backtracking and 0.22 for MRV, so time grows with difficulty for all algorithms, but with a very large scatter for plain backtracking.

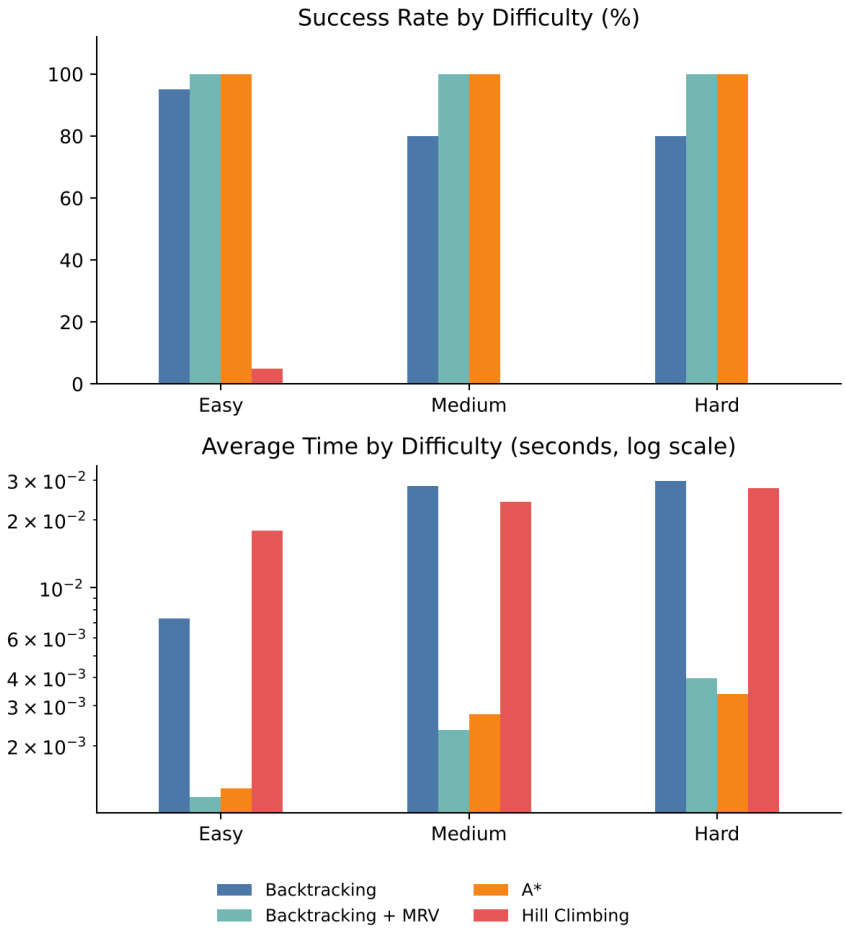


Figure 11. Success rate (top) and average time (bottom, log scale) by difficulty level.

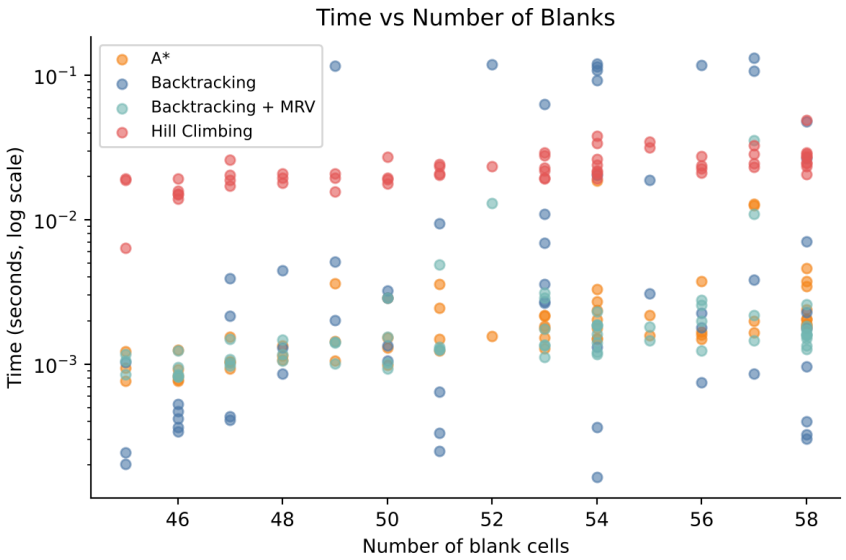


Figure 12. Running time against the number of blank cells for every run (log scale).

Summary of each algorithm.

- **Backtracking** is complete in principle, but without an ordering heuristic it can spend tens of thousands of assignments in a bad region of the search space and gave up on 9 of 60 puzzles within the cap.

- **Backtracking with MRV** keeps the simplicity and the low memory of backtracking, and the fail-first ordering reduces the work by a factor of about 99 in states and about 9 in time.
- **A*** reaches the same efficiency by a different route: the heuristic orders the states and detects dead ends, so it also fills a cell for fewer than two generated states. It needs more memory than hill climbing, but less than the recursive versions on these puzzles.
- **Hill climbing** is fast per step but does not use the digits already placed in a smart way; it minimises conflicts and stops in local minima. It is the only algorithm that is not complete, and its success rate is close to zero.

5.7 Visualization of Search Process

Figure 13 visualises the search on puzzle 6. The top chart shows hill climbing: in each of the ten restarts the cost falls quickly from about 40 repeated digits, but every restart ends at a local minimum (cost 6 to 16) and the next restart starts again at a high cost. The bottom chart shows A*: the f value of the popped state falls steadily from 280 to 58 (the number of blanks) while the heap grows slowly to a maximum of 18 states; the search moves almost directly to the goal.

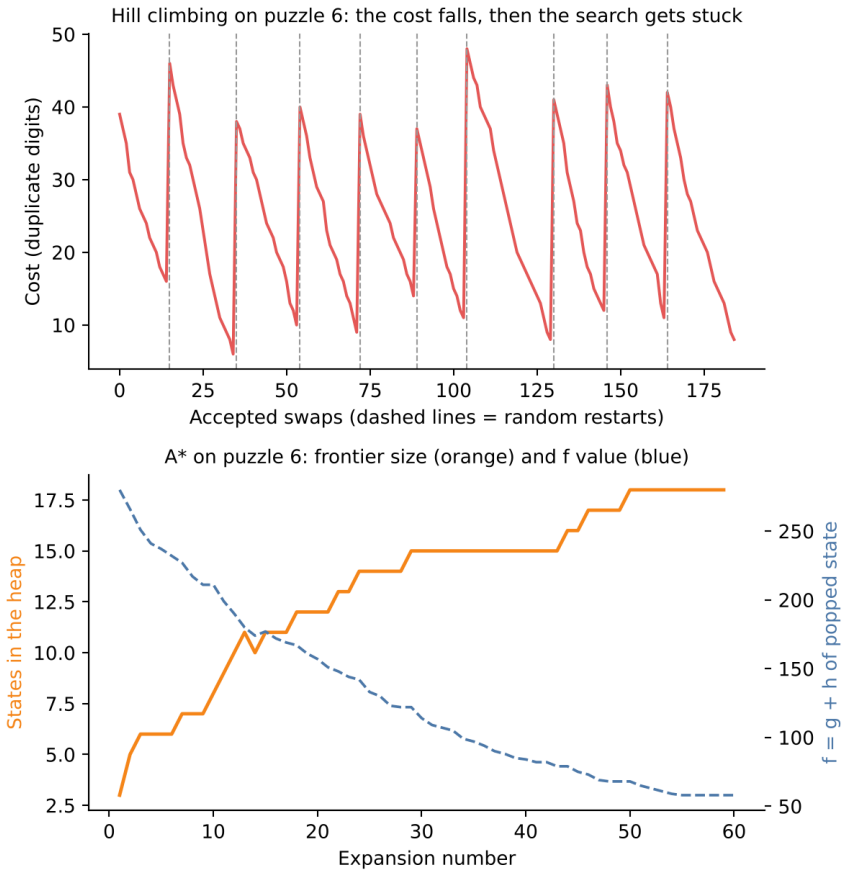


Figure 13. Search process on puzzle 6: hill-climbing cost per accepted swap (top) and A* heap size and f value per expansion (bottom).

Together with the search tree in Figure 6 and the heap trace in Table 7, these charts show the difference between the algorithms: A* and MRV use the constraints to decide *what to try next*, while hill climbing only measures how wrong the current board is.

6. Conclusion

6.1 Summary of the Project and Its Impact

Backtracking (plain and with MRV), A* search and hill climbing were implemented and tested on the same 60 Sudoku puzzles from three difficulty groups. Backtracking with MRV and A* solved every puzzle in about 2.5 ms using about 100 states, plain backtracking solved 85 % and needed about 99 times more states, and hill climbing solved 1.7 %. Under the notebook's ranking rule A* is first, but it is statistically tied with backtracking with MRV; both are far ahead of the other two.

The main finding is that the algorithms that use the constraints directly win. Sudoku is a constraint satisfaction problem, and the two methods that look at the legal values of the empty cells (MRV and the A* heuristic) avoid wrong branches almost entirely. Hill climbing treats the constraints only as a score to minimise and gets trapped. The same lesson applies to other constraint problems such as timetabling and scheduling. The measurements in this report come from a locally generated puzzle file with the same format as the Kaggle file (Section 4.1); running the notebook on the original file will give the final numbers for the submission.

6.2 Reflection on What Was Learned

- Completeness and speed are different properties: plain backtracking is complete but not reliable under a time cap, while hill climbing is fast per step but incomplete.
- A good ordering heuristic can matter more than the choice of algorithm; adding MRV to backtracking made it about as fast as A*.
- A heuristic does not have to be admissible to be useful, but then its optimality guarantee has to be justified in another way. Here it holds because all solutions have the same cost.
- Fair experiments need identical inputs, an independent correctness check and effort caps, and results must be examined for noise: the first place of A* disappeared when the timing was repeated.
- Not all expectations written before an experiment are confirmed by it (for example the memory use of A*), which is why the numbers have to be read before the conclusions are written.

References

[1] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.

[2] R. M. Haralick and G. L. Elliott, "Increasing tree search efficiency for constraint satisfaction problems," *Artif. Intell.*, vol. 14, no. 3, pp. 263–313, 1980.

[3] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, 2007.

[4] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Sci. Conf.*, 2010, pp. 56–61.

[5] Python Software Foundation, "heapq: Heap queue algorithm," Python 3 Documentation. [Online]. Available: https://docs.python.org/3/library/heapq.html

[6] R. Rao, "Sudoku [9 million puzzles; Data set]," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/rohanrao/sudoku

[7] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.

[8] T. Yato and T. Seta, "Complexity and completeness of finding another solution and its application to puzzles," *IEICE Trans. Fundamentals*, vol. E86-A, no. 5, pp. 1052–1060, 2003.

Appendices

Appendix A. Full Code Listing of the Notebook

The notebook code in the order in which it is run. Cell numbers refer to the code cells of sudoku_ai_analysis.ipynb.

Code cell 1: Imports and plot settings

```
import time
import heapq
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi']=110
plt.rcParams['axes.spines.top']=False
plt.rcParams['axes.spines.right']=False
```

Code cell 2: Loading the data

```
N_PUZZLES=60
KEEP=0.003
rng=random.Random(42)
df=pd.read_csv('sudoku.csv',dtype=str,skiprows=lambda i:i>0 and rng.random()>KEEP)
df=df.rename(columns={df.columns[0]:'quizzes',df.columns[1]:'solutions'})
df['quizzes']=df['quizzes'].str.replace('.','0',regex=False)
sample=df.sample(N_PUZZLES,random_state=42).reset_index(drop=True)
```

```
sample['blanks']=sample['quizzes'].str.count('0')
sample['level']=pd.qcut(sample['blanks'].rank(method='first'),3,labels=['Easy','Medium','Hard']).astype(str)
sample.groupby('level')['blanks'].agg(['count','min','max','mean'])
```

Code cell 3: Board helpers

```
ROWS=[[r*9+c for c in range(9)] for r in range(9)]
COLS=[[r*9+c for r in range(9)] for c in range(9)]
BOXES=[[(br*3+i)*9+bc*3+j for i in range(3) for j in range(3)] for br in range(3) for bc in range(3)]
PEERS=[[j for j in range(81) if j!=i and (j//9==i//9 or j%9==i%9 or (j//27==i//27 and (j%9)//3==(i%9)//3))] for i in range(81)]
def candidates(g,i):
    used={g[j] for j in PEERS[i]}
    return [v for v in range(1,10) if v not in used]
def to_grid(s):
    return [int(ch) for ch in s]
```

Code cell 4: Backtracking

```
def backtracking(puzzle,mrv=False,limit=50000):
    g=list(puzzle)
    count=[0]
    def solve():
        empties=[i for i in range(81) if g[i]==0]
        if not empties:
            return True
        if count[0]>=limit:
            return False
        i=min(empties,key=lambda k:len(candidates(g,k))) if mrv else empties[0]
        for v in candidates(g,i):
            g[i]=v
            count[0]+=1
            if solve():
                return True
        g[i]=0
        return False
    ok=solve()
    return ok,count[0],g
```

Code cell 5: A search*

```
def analyse(g):
    total=0
    best=None
    best_c=None
    for i in range(81):
        if g[i]==0:
            c=candidates(g,i)
            if not c:
                return None
            total+=len(c)
            if best is None or len(c)<len(best_c):
                best,best_c=i,c
    return total,best,best_c
def astar(puzzle,limit=3000):
    start=tuple(puzzle)
    info=analyse(start)
    if info is None:
        return False,0,list(start)
    tie=0
    heap=[(info[0],tie,0,start,info)]
    nodes=0
    expanded=0
    while heap and expanded<limit:
        f,_,g,state,info=heapq.heappop(heap)
        if info[1] is None:
            return True,nodes,list(state)
        expanded+=1
        for v in info[2]:
            child=list(state)
            child[info[1]]=v
            nodes+=1
            ci=analyse(child)
            if ci is None:
                continue
            tie+=1
            heapq.heappush(heap,(g+1+ci[0],tie,g+1,tuple(child),ci))
    return False,nodes,list(start)
```

Code cell 6: Hill climbing

```
def line_cost(g,idx):
    return 9-len({g[k] for k in idx})
def total_cost(g):
```

```
    return sum(line_cost(g,l) for l in ROWS+COLS)
def hill_climbing(puzzle,restarts=10,max_steps=300):
    free=[[k for k in box if puzzle[k]==0] for box in BOXES]
    evaluated=0
    g=list(puzzle)
    for _ in range(restarts):
        g=list(puzzle)
        for b,box in enumerate(BOXES):
            missing=[v for v in range(1,10) if v not in {g[k] for k in box}]
            random.shuffle(missing)
            for k in free[b]:
                g[k]=missing.pop()
        cost=total_cost(g)
        for _ in range(max_steps):
            if cost==0:
                return True,evaluated,g
            moved=False
            for b in random.sample(range(9),9):
                f=free[b]
                best=0
                move=None
                for x in range(len(f)):
                    for y in range(x+1,len(f)):
                        a,c=f[x],f[y]
                        rows={a//9,c//9}
                        cols={a%9,c%9}
                        before=sum(line_cost(g,ROWS[r]) for r in rows)+sum(line_cost(g,COLS[k]) for k in cols)
                        g[a],g[c]=g[c],g[a]
                        after=sum(line_cost(g,ROWS[r]) for r in rows)+sum(line_cost(g,COLS[k]) for k in cols)
                        g[a],g[c]=g[c],g[a]
                        evaluated+=1
                        if after-before<best:
                            best=after-before
                            move=(a,c)
                if move:
                    a,c=move
                    g[a],g[c]=g[c],g[a]
                    cost+=best
                    moved=True
                    break
            if not moved:
                break
        if cost==0:
            return True,evaluated,g
    return False,evaluated,g
```

Code cell 7: Running the experiment

```
def is_valid(grid,puzzle):
    full=set(range(1,10))
    return all(set(grid[k] for k in line)==full for line in ROWS+COLS+BOXES) and all(p==0 or p==v for p,v in zip(puzzle,grid))
random.seed(42)
algorithms={
    'Backtracking':lambda p:backtracking(p,mrv=False),
    'Backtracking + MRV':lambda p:backtracking(p,mrv=True),
    'A*':astar,
    'Hill Climbing':hill_climbing
}
records=[]
for idx,row in sample.iterrows():
    puzzle=to_grid(row['quizzes'])
    for name,fn in algorithms.items():
        t=time.perf_counter()
        ok,nodes,grid=fn(puzzle)
        elapsed=time.perf_counter()-t
        records.append({'puzzle':idx,'algorithm':name,'solved':bool(ok                                                                                    and
is_valid(grid,puzzle)),'time':elapsed,'nodes':nodes,'blanks':row['blanks'],'level':row['level']})
results=pd.DataFrame(records)
summary=results.groupby('algorithm').agg(success_rate=('solved',lambda
s:s.mean()*100),mean_time=('time','mean'),median_time=('time','median'),max_time=('time','max'),mean_nodes=('nodes','mean')).reindex(list(al
gorithms))
summary.round(4)
```

Code cell 8: Overall comparison charts

```
colors=['#4C78A8','#72B7B2','#F58518','#E45756']
fig,ax=plt.subplots(1,2,figsize=(13,4.5))
ax[0].bar(summary.index,summary['success_rate'],color=colors)
ax[0].set_title('Success Rate (%)')
ax[0].set_ylim(0,110)
for i,v in enumerate(summary['success_rate']):
    ax[0].text(i,v+2,f'{v:.0f}%',ha='center')
ax[1].bar(summary.index,summary['mean_time'],color=colors)
ax[1].set_yscale('log')
ax[1].set_title('Average Time per Puzzle (seconds, log scale)')
for a in ax:
    a.tick_params(axis='x',rotation=20)
```

```
plt.tight_layout()
plt.show()
```

Code cell 9: Search effort and time spread charts

```
fig,ax=plt.subplots(1,2,figsize=(13,4.5))
ax[0].bar(summary.index,summary['mean_nodes'],color=colors)
ax[0].set_yscale('log')
ax[0].set_title('Average States Explored (log scale)')
ax[0].tick_params(axis='x',rotation=20)
data=[results[results['algorithm']==a]['time'] for a in summary.index]
bp=ax[1].boxplot(data,patch_artist=True)
for patch,c in zip(bp['boxes'],colors):
    patch.set_facecolor(c)
ax[1].set_xticklabels(summary.index,rotation=20)
ax[1].set_yscale('log')
ax[1].set_title('Time Distribution (seconds, log scale)')
plt.tight_layout()
plt.show()
```

Code cell 10: Effect of difficulty: bar charts

```
order=['Easy','Medium','Hard']
success=results.pivot_table(index='level',columns='algorithm',values='solved',aggfunc='mean').reindex(order)[list(algorithms)]*100
avg_time=results.pivot_table(index='level',columns='algorithm',values='time',aggfunc='mean').reindex(order)[list(algorithms)]
fig,ax=plt.subplots(1,2,figsize=(13,4.5))
success.plot(kind='bar',ax=ax[0],color=colors,rot=0)
ax[0].set_title('Success Rate by Difficulty (%)')
ax[0].set_xlabel('')
ax[0].set_ylim(0,110)
avg_time.plot(kind='bar',ax=ax[1],color=colors,rot=0,logy=True)
ax[1].set_title('Average Time by Difficulty (seconds, log scale)')
ax[1].set_xlabel('')
plt.tight_layout()
plt.show()
```

Code cell 11: Effect of difficulty: time against blanks

```
fig,ax=plt.subplots(figsize=(9,5))
for name,grp in results.groupby('algorithm'):
    ax.scatter(grp['blanks'],grp['time'],label=name,alpha=0.6,s=28,color=colors[list(algorithms).index(name)])
ax.set_yscale('log')
ax.set_xlabel('Number of blank cells')
ax.set_ylabel('Time (seconds, log scale)')
ax.set_title('Time vs Number of Blanks')
ax.legend()
plt.tight_layout()
plt.show()
```

Code cell 12: Ranking

```
ranking=summary.sort_values(['success_rate','mean_time'],ascending=[False,True])
ranking['rank']=range(1,len(ranking)+1)
display(ranking[['rank','success_rate','mean_time','mean_nodes']].round(4))
print('Best algorithm:',ranking.index[0])
```

Appendix B. Supplementary Measurement Code

The notebook does not measure memory and runs each algorithm once per puzzle. The two scripts below were used for the peak-memory values in Table 13 and for the repeated timing in Table 17. They use the functions of the notebook unchanged.

Peak memory per run

```
import tracemalloc
mem = []
random.seed(42)
for idx, row in sample.iterrows():
    puzzle = to_grid(row['quizzes'])
    for name, fn in algorithms.items():
        tracemalloc.start()
        fn(puzzle)
        current, peak = tracemalloc.get_traced_memory()
        tracemalloc.stop()
        mem.append({'puzzle': idx, 'algorithm': name,
                    'peak_kb': peak / 1024, 'level': row['level']})
```

```
mem = pd.DataFrame(mem)
mem.groupby('algorithm')['peak_kb'].agg(['mean', 'median', 'max'])
```

*Repeated timing of Backtracking + MRV and A**

```
for rep in range(8):
    total = {'mrv': 0, 'astar': 0}
    for _, r in sample.iterrows():
        p = to_grid(r['quizzes'])
        t = time.perf_counter()
        backtracking(p, mrv=True)
        total['mrv'] += time.perf_counter() - t
        t = time.perf_counter()
        astar(p)
        total['astar'] += time.perf_counter() - t
    print(rep + 1, total['mrv'] / 60, total['astar'] / 60)
```

Appendix C. Reproducing the Results

To obtain the final numbers with the original dataset, download sudoku.csv from Kaggle, place it in the folder of the notebook, run all cells, and replace the values in the tables and the figures of Section 5 with the notebook output. Table 10 lists where each result is produced.

Table 18. Where each result of Section 5 comes from.

Item in this report	Produced by
Table 11 and Figure 8	Notebook sections 6 and 7 (summary, success-rate and time charts).
Figure 9	Notebook section 8 (states explored, time distribution).
Table 12, Figure 11 and Figure 12	Notebook section 9 (effect of difficulty).
Table 16	Notebook section 10 (ranking).
Table 13, Figure 10, Table 17	Supplementary scripts in Appendix B (not part of the notebook).
Figure 1, Figure 6, Table 7, Figure 13	Instrumented copies of analyse/astar and hill_climbing run on puzzle 6 (same logic, extra logging).